

PROJECT REPORT

Spring 2026

Artificial Intelligence Lab (AIL-201)



PROJECT TITLE: Automatic Timetable Generator

(CSP Based)

BS(AI)-4-A

Group Members

Name	Enrollment
1. Haroon Rashid	02-136242-042
2. Ali Asad	02-136242-006
3. Ishraq Sajid	02-136242-040

Submitted to:

Miss Zahida Naz

BAHRIA UNIVERSITY KARACHI CAMPUS

Department of Computer Science

Table of Contents

ABSTRACT	3
INTRODUCTION.....	4
PROJECT SCOPE	5
PROBLEM STATEMENT	6
METHODOLOGY	7
FUTURE DEVELOPMENT	8
CONCLUSION	9
Code	10
Output.....	19

ABSTRACT

Creating a university timetable by hand is one of those tasks that sounds simple on the surface but quickly turns into a massive headache. Between keeping track of which teacher is available when, making sure no two classes share the same room, and ensuring labs get their full three-hour blocks, there are hundreds of small decisions that all have to work together. This project presents an Automated Timetable Generator built specifically for the BS Artificial Intelligence program, covering all four semesters. The system takes the list of subjects, teachers, and rooms as input and uses a randomized constraint-satisfaction algorithm to produce a complete, conflict-free weekly schedule. It also comes with a simple graphical interface so users can adjust the working days and time slots without touching any code. The result is a tool that can generate a valid timetable in seconds, something that would otherwise take hours of manual effort.

INTRODUCTION

Every university, at the start of each semester, faces the same challenge: fitting dozens of courses, teachers, rooms, and time slots together in a way that works for everyone. For small departments this might be manageable with a spreadsheet, but as the number of classes and constraints grows, it becomes increasingly error-prone and time-consuming.

This project currently runs four active batches (BSAI-1 through BSAI-4), each with its own set of theory courses and lab sessions. Labs are especially tricky because they need to be scheduled as uninterrupted three-hour blocks, they require specific rooms, and the teacher is often the same person handling the corresponding theory course. On top of that, no teacher can be in two places at once, and no room can host two classes simultaneously.

This project was built to solve exactly that problem. Rather than relying on manual scheduling, the system automates the entire process. The user simply picks their working days and time slot length, clicks a button, and the software figures out a valid schedule for all four batches at once. If a valid arrangement exists within the given constraints, the system will find it.

PROJECT SCOPE:

The scope of this project covers the following:

- All four semesters of the BSAI program (BSAI-1, BSAI-2, BSAI-3, BSAI-4)
- A total of 40+ subjects across theory and lab categories
- 33 theory classrooms across three floors (E-101 to E-311)
- 9 specialized lab rooms including the AI Lab, DLD Lab, Web Lab, and CN Lab
- Teacher conflict detection: no teacher is double-booked at any point
- Room conflict detection: no room is assigned two classes at the same time
- Correct lab-to-room mapping: each lab subject is only assigned to its permitted room(s)
- Configurable working days (any combination of Mon–Sun)
- Configurable day start time, end time, and slot duration (30, 60, or 90 minutes)
- A graphical interface for non-technical users

The system does not currently handle student-level conflicts (e.g., elective clashes), teacher preferences, or multi-week rotating schedules. These are areas that could be added in future versions.

PROBLEM STATEMENT:

Manually scheduling classes for a department with multiple batches, dozens of subjects, and limited shared resources (rooms and teachers) is a complex combinatorial problem. The core difficulty is that every scheduling decision affects multiple other decisions. Placing a lab session for BSAI-3 in the CN Lab at 9:00 AM on Monday means that room is now unavailable to everyone else for the next three hours. Assigning Dr. Kamran to a 10:00 AM theory slot means he cannot simultaneously conduct his lab session.

When done manually, these constraints are easy to violate by accident. A coordinator working with a spreadsheet might not immediately notice that a teacher has been scheduled for two back-to-back sessions in different buildings, or that a specialized lab has been double-booked. Finding and fixing these errors takes time, and the problem gets worse as the schedule gets more packed.

The problem this project addresses can be stated clearly: given a fixed set of classes, teachers, rooms, and time constraints, how do we automatically generate a valid weekly schedule that satisfies all hard constraints (no conflicts) while making practical use of available time slots? The answer needs to work reliably and quickly, even when the input is dense and the number of possible arrangements is enormous.

METHODOLOGY:

The system works in two parts: a scheduling engine that builds the timetable, and a graphical interface that lets users control it.

The scheduling engine uses the following approach:

- All subjects across all four batches are collected into a single task list and then shuffled randomly. This randomness is important because it means different runs of the program will try different orderings, which helps avoid getting stuck.
- For each subject, the algorithm tries to find an available time slot by iterating through days and slots in random order. For theory subjects, it tries to place one slot at a time (with a rule that prevents the same subject from appearing in more than two consecutive slots on a single day). For lab subjects, it tries to place a single uninterrupted 3-hour block.
- Before placing any subject, the system checks three things: the class's own slot is free, the teacher is not busy elsewhere at that time, and the room is not already taken. Only if all three conditions are satisfied does the booking go through.
- Each lab subject has a defined list of permitted rooms. For example, DCN labs can only go into the CN Lab, while general programming labs can use any of the five general-purpose labs or the AI Lab.
- If the algorithm cannot place a subject after exhausting all day/slot/room combinations, the entire attempt is discarded and a fresh attempt begins from scratch. The system tries up to 200 times before giving up and reporting that no valid schedule was found.

The graphical interface was built with Python's Tkinter library. It presents a sidebar where the user can check off days to exclude, set the start and end times of the working day, and choose a slot length. When the user clicks the generate button, the results are displayed as color-coded tables — blue for theory classes and green for lab sessions — with each table representing one batch.

FUTURE DEVELOPMENT

The current version of the system works well for its intended purpose, but there are several directions it could grow in:

- **Teacher preferences:** Right now, the algorithm treats all time slots equally. A future version could allow teachers to mark preferred or unavailable slots, and the scheduler would try to respect those preferences while still satisfying hard constraints.
- **Export functionality:** The timetable is currently only visible within the application window. Adding the ability to export schedules to Excel, PDF, or a printable format would make the tool much more practical for real departmental use.
- **Smarter algorithms:** The current approach is essentially a randomized brute-force search. Replacing it with a proper constraint propagation algorithm (such as backtracking with forward checking) or a genetic algorithm would make it faster and more likely to find a solution on the first try.
- **Multi-department support:** The data (classes, teachers, rooms) is currently hard-coded. Extending the system to read from a database or configuration file would allow it to handle multiple departments without any code changes.
- **Web-based interface:** Moving from a desktop Tkinter app to a web application would make the tool accessible from any device and easier to share across a department.
- **Student elective conflict handling:** For programs where students choose elective courses, the scheduler could be extended to check that no student's chosen electives clash with each other.

CONCLUSION

This project sets out to solve a practical, everyday problem in academic administration: the creation of a conflict-free class schedule for a multi-batch university program. The result is a working desktop application that automates the entire process.

The system correctly handles the specific constraints of the BSAI program, dedicated lab rooms, three-hour lab blocks, shared teachers across batches, and many theory classrooms. Users can customize the schedule around their working week and available hours, and the tool will produce a valid timetable in seconds.

While there is room to grow smarter algorithms, export features, a web interface — the core functionality is solid and demonstrates the real-world value of applying basic constraint satisfaction techniques to a scheduling problem. What used to be a tedious manual exercise can now be handled automatically, freeing up time and reducing the chance of errors.

Overall, this project is a good example of how a relatively small piece of software, built with the right logic, can make a meaningful difference in how an organization manages its resources.

Code

```
import tkinter as tk
from tkinter import ttk
import random

THEORY_ROOMS = []
for number in range(101, 112):
    THEORY_ROOMS.append("E-" + str(number))
for number in range(201, 212):
    THEORY_ROOMS.append("E-" + str(number))
for number in range(301, 312):
    THEORY_ROOMS.append("E-" + str(number))

LAB_ROOMS = ["Lab 1", "Lab 2", "Lab 3", "Lab 4", "Lab 5", "AI Lab", "DLD Lab",
"Web Lab", "CN Lab"]

LAB_MAPPING = {
    "AP-L(A)": ["AP Lab"],
    "AP-L(B)": ["AP Lab"],
    "CP-L": ["Lab 1", "Lab 2", "Lab 3", "Lab 4", "Lab 5", "AI Lab"],
    "IICT-L": ["Lab 1", "Lab 2", "Lab 3", "Lab 4", "Lab 5", "AI Lab"],
    "OOP-L": ["Lab 1", "Lab 2", "Lab 3", "Lab 4", "Lab 5", "AI Lab"],
    "DD-L(A)": ["DLD Lab"],
    "DD-L(B)": ["DLD Lab"],
    "COAL-L": ["Lab 1", "Lab 2", "Lab 3", "Lab 4", "Lab 5", "AI Lab"],
    "PAI-L": ["Lab 1", "Lab 2", "Lab 3", "Lab 4", "Lab 5", "AI Lab"],
    "DSA-L": ["Lab 1", "Lab 2", "Lab 3", "Lab 4", "Lab 5", "AI Lab"],
    "DBMS-L": ["Web Lab"],
    "DCN-L(A)": ["CN Lab"],
    "DCN-L(B)": ["CN Lab"],
    "AI-L": ["AI Lab"],
}

CLASSES = {
    "BSAI-1": {
        "PPE": {"teacher": "Dr. Hassan", "hours_needed": 2, "type": "theory"},
        "IICT-L": {"teacher": "Dr. Imran", "hours_needed": 3, "type": "lab"},
        "IICT": {"teacher": "Ms. Munazza", "hours_needed": 3, "type": "theory"},
        "DM": {"teacher": "Ms. Munazza", "hours_needed": 3, "type": "theory"},
        "CP-L": {"teacher": "Ms. Zahida", "hours_needed": 3, "type": "lab"},
        "Tajweed": {"teacher": "Dr. Nawaz", "hours_needed": 1, "type":
"theory"},
        "AP": {"teacher": "Dr. Ali", "hours_needed": 3, "type": "theory"},
    }
```

```

    "AP-L(A)": {"teacher": "Sir Fahad", "hours_needed": 3, "type": "lab"},
    "AP-L(B)": {"teacher": "Sir Anas", "hours_needed": 3, "type": "lab"},
    "CP": {"teacher": "Ms. Salas", "hours_needed": 3, "type": "theory"},
    "IS": {"teacher": "Sir Anwar", "hours_needed": 2, "type": "theory"},
},
"BSAI-2": {
    "DD": {"teacher": "Ms. Amna", "hours_needed": 3, "type": "theory"},
    "PS": {"teacher": "Engr. Reema", "hours_needed": 3, "type": "theory"},
    "OOP-L": {"teacher": "Dr. Imran", "hours_needed": 3, "type": "lab"},
    "OOP": {"teacher": "Sir Ali", "hours_needed": 3, "type": "theory"},
    "DD-L(A)": {"teacher": "Ms. Amna", "hours_needed": 3, "type": "lab"},
    "DD-L(B)": {"teacher": "Sir Haroon", "hours_needed": 3, "type": "lab"},
    "FE": {"teacher": "Sir Chauhan", "hours_needed": 3, "type": "theory"},
    "PST": {"teacher": "Ms. Farah", "hours_needed": 2, "type": "theory"},
    "UQ-1": {"teacher": "Sir Muneeb", "hours_needed": 1, "type": "theory"},
    "AC&AG": {"teacher": "Ms. Sadia", "hours_needed": 3, "type": "theory"},
},
"BSAI-3": {
    "DSA": {"teacher": "Dr. Bilal", "hours_needed": 3, "type": "theory"},
    "DSA-L": {"teacher": "Dr. Bilal", "hours_needed": 3, "type": "lab"},
    "COAL": {"teacher": "Sir Asim", "hours_needed": 2, "type": "theory"},
    "COAL-L": {"teacher": "Sir Asim", "hours_needed": 3, "type": "lab"},
    "PAI": {"teacher": "Dr. Sana", "hours_needed": 3, "type": "theory"},
    "PAI-L": {"teacher": "Dr. Sana", "hours_needed": 3, "type": "lab"},
    "LA": {"teacher": "Dr. Kamran", "hours_needed": 3, "type": "theory"},
    "CS": {"teacher": "Sir Noman", "hours_needed": 2, "type": "theory"},
    "UQ-2": {"teacher": "Dr. Nawaz", "hours_needed": 1, "type": "theory"},
    "CCE": {"teacher": "Ms. Salas", "hours_needed": 2, "type": "theory"},
},
"BSAI-4": {
    "DBMS": {"teacher": "Dr. Ayesha", "hours_needed": 3, "type": "theory"},
    "DBMS-L": {"teacher": "Dr. Ayesha", "hours_needed": 3, "type": "lab"},
    "ENT": {"teacher": "Ms. Farah", "hours_needed": 2, "type": "theory"},
    "AI": {"teacher": "Dr. Bilal", "hours_needed": 3, "type": "theory"},
    "AI-L": {"teacher": "Dr. Bilal", "hours_needed": 3, "type": "lab"},
    "CT": {"teacher": "Sir Noman", "hours_needed": 3, "type": "theory"},
    "DCN": {"teacher": "Dr. Kamran", "hours_needed": 3, "type": "theory"},
    "DCN-L(A)": {"teacher": "Dr. Kamran", "hours_needed": 3, "type": "lab"},
    "DCN-L(B)": {"teacher": "Dr. Kamran", "hours_needed": 3, "type": "lab"},
    "UQ-3": {"teacher": "Dr. Nawaz", "hours_needed": 1, "type": "theory"},
}
}

def generate_timetable_logic(active_days, total_slots_available):
    for attempt in range(200):

```

```

timetable = {}
for class_name in CLASSES:
    day_map = {}
    for day_name in active_days:
        day_map[day_name] = [None] * total_slots_available
    timetable[class_name] = day_map

teacher_busy_status = {}
room_busy_status = {}

# --- DYNAMIC TARGET DAYS OFF CONFIGURATION ---
# For each class, pre-determine which days we WANT to use and which days
we WANT TO AVOID.
class_day_priorities = {}
for class_name in CLASSES:
    if len(active_days) >= 6:
        days_off_target = 2 # Try to give 2 days off if week layout is
large
    elif len(active_days) >= 4:
        days_off_target = 1 # Try to give 1 day off for standard 4-5
active days
    else:
        days_off_target = 0

    shuffled_active = random.sample(active_days, len(active_days))
    if days_off_target > 0:
        preferred = shuffled_active[:-days_off_target]
        avoided = shuffled_active[-days_off_target:]
    else:
        preferred = shuffled_active
        avoided = []
    class_day_priorities[class_name] = (preferred, avoided)
# -----

tasks_to_schedule = []
for class_name, subjects_dict in CLASSES.items():
    for subject_name, subject_info in subjects_dict.items():
        tasks_to_schedule.append((class_name, subject_name,
subject_info))

random.shuffle(tasks_to_schedule)

def check_if_safe(class_name, subject_info, day, start_slot, room,
duration):
    if start_slot + duration > total_slots_available:

```

```

        return False

    current_teacher = subject_info["teacher"]
    for current_slot in range(start_slot, start_slot + duration):
        if timetable[class_name][day][current_slot] is not None:
            return False
        if (current_teacher, day, current_slot) in teacher_busy_status:
            return False
        if (room, day, current_slot) in room_busy_status:
            return False
    return True

def mark_as_booked(class_name, subject_name, subject_info, day,
start_slot, room, duration):
    current_teacher = subject_info["teacher"]
    for current_slot in range(start_slot, start_slot + duration):
        timetable[class_name][day][current_slot] = (subject_name, room)
        teacher_busy_status[(current_teacher, day, current_slot)] = True
        room_busy_status[(room, day, current_slot)] = True

overall_success_flag = True
for class_name, subject_name, subject_info in tasks_to_schedule:
    is_lab_class = (subject_info["type"] == "lab")

    if is_lab_class:
        schedule_blocks = [3]
    else:
        hours = subject_info["hours_needed"]
        if hours == 1:
            schedule_blocks = [1]
        elif hours == 2:
            schedule_blocks = [2]
        elif hours == 3:
            schedule_blocks = [2, 1]
        elif hours == 4:
            schedule_blocks = [2, 2]
        else:
            schedule_blocks = [1] * hours

    used_days_for_subject = []

    for current_duration in schedule_blocks:
        subject_was_placed = False

        # --- COMPACTED DAY SELECTION ---

```

```

        # Retrieve the preferred and avoided day tiers for this specific
class.
        # Shuffle internally so there's still local variation, but
preferred always come first.
        pref_days, avoid_days = class_day_priorities[class_name]
        shuffled_days = random.sample(pref_days, len(pref_days)) +
random.sample(avoid_days, len(avoid_days))
        # -----

        shuffled_slots = random.sample(range(total_slots_available),
total_slots_available)

        for day in shuffled_days:
            if len(schedule_blocks) > 1 and current_duration == 1 and
day in used_days_for_subject:
                continue

            allowed_rooms = LAB_MAPPING.get(subject_name, LAB_ROOMS) if
is_lab_class else THEORY_ROOMS
            shuffled_rooms = random.sample(allowed_rooms,
len(allowed_rooms))

            for slot in shuffled_slots:
                for room in shuffled_rooms:
                    if check_if_safe(class_name, subject_info, day,
slot, room, current_duration):
                        mark_as_booked(
                            class_name,
                            subject_name,
                            subject_info,
                            day,
                            slot,
                            room,
                            current_duration
                        )
                        used_days_for_subject.append(day)
                        subject_was_placed = True
                        break
                    if subject_was_placed:
                        break
                if subject_was_placed: break

        if not subject_was_placed:
            overall_success_flag = False
            break

```

```

        if not overall_success_flag: break

    if overall_success_flag:
        return timetable
    return None

class SimpleTimetableUI:
    def __init__(self, main_window):
        self.main_window = main_window
        self.main_window.title("Timetable Generator")
        self.main_window.geometry("1300x850")

        time_options = []
        for hour in range(24):
            time_options.append(f"{hour:02d}:00")
            time_options.append(f"{hour:02d}:30")

        sidebar_frame = ttk.Frame(main_window, padding=20)
        sidebar_frame.pack(side="left", fill="y")

        ttk.Label(sidebar_frame, text="SETTINGS", font=("Arial", 14,
"bold")).pack(pady=10)

        ttk.Label(sidebar_frame, text="Exclude These Days:").pack(anchor="w")
        self.day_names = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]
        self.exclude_check_variables = {}
        for day in self.day_names:
            check_var = tk.BooleanVar(value=False)
            self.exclude_check_variables[day] = check_var
            ttk.Checkbutton(sidebar_frame, text=day,
variable=check_var).pack(anchor="w")

        ttk.Label(sidebar_frame, text="\nDay Starts At:").pack(anchor="w")
        self.start_time_dropdown = ttk.Combobox(sidebar_frame,
values=time_options, state="readonly")
        self.start_time_dropdown.set("08:30")
        self.start_time_dropdown.pack(fill="x")

        ttk.Label(sidebar_frame, text="Day Ends At:").pack(anchor="w")
        self.end_time_dropdown = ttk.Combobox(sidebar_frame,
values=time_options, state="readonly")
        self.end_time_dropdown.set("16:30")
        self.end_time_dropdown.pack(fill="x")

        ttk.Label(sidebar_frame, text="Slot Length (Minutes):").pack(anchor="w")

```

```

        self.slot_length_dropdown = ttk.Combobox(sidebar_frame, values=[30, 60,
90], state="readonly")
        self.slot_length_dropdown.set(60)
        self.slot_length_dropdown.pack(fill="x")

        ttk.Button(sidebar_frame, text="GENERATE SCHEDULE",
command=self.handle_generate_click).pack(pady=30, fill="x")

        self.display_canvas = tk.Canvas(main_window, bg="white")
        self.display_canvas.pack(side="left", fill="both", expand=True)

        scrollbar = ttk.Scrollbar(main_window, orient="vertical",
command=self.display_canvas.yview)
        scrollbar.pack(side="right", fill="y")
        self.display_canvas.configure(yscrollcommand=scrollbar.set)

        self.internal_scroll_frame = ttk.Frame(self.display_canvas)
        self.display_canvas.create_window((0, 0),
window=self.internal_scroll_frame, anchor="nw")
        self.internal_scroll_frame.bind("<Configure>", lambda event:
self.display_canvas.configure(scrollregion=self.display_canvas.bbox("all")))

    def handle_generate_click(self):
        for child in self.internal_scroll_frame.winfo_children():
            child.destroy()

        active_days_list = []
        for day in self.day_names:
            if self.exclude_check_variables[day].get() == False:
                active_days_list.append(day)

        if not active_days_list:
            tk.Label(self.internal_scroll_frame, text="Error: Please keep at
least one day active!").pack()
            return

    def get_total_minutes(time_string):
        parts = time_string.split(":")
        return int(parts[0]) * 60 + int(parts[1])

    start_total_minutes = get_total_minutes(self.start_time_dropdown.get())
    end_total_minutes = get_total_minutes(self.end_time_dropdown.get())
    minutes_per_slot = int(self.slot_length_dropdown.get())

    available_minutes = end_total_minutes - start_total_minutes

```

```

        total_slots_needed = available_minutes // minutes_per_slot

        if total_slots_needed <= 0:
            tk.Label(self.internal_scroll_frame, text="Error: End time must be
later than start time!").pack()
            return

        time_header_labels = []
        for index in range(total_slots_needed):
            minutes_at_this_slot = start_total_minutes + (index *
minutes_per_slot)
            hour_display = minutes_at_this_slot // 60
            minute_display = minutes_at_this_slot % 60
            time_header_labels.append(f"{hour_display:02d}:{minute_display:02d}")
    )

    final_timetable = generate_timetable_logic(active_days_list,
total_slots_needed)

    if final_timetable is None:
        tk.Label(self.internal_scroll_frame, text="Could not find a valid
schedule. Try adding more time or days.").pack(pady=20)
        return

    for class_name, days_data in final_timetable.items():
        tk.Label(self.internal_scroll_frame, text=f"SCHEDULE FOR:
{class_name}", font=("Arial", 14, "bold")).pack(pady=(25, 5))

        table_container = ttk.Frame(self.internal_scroll_frame)
        table_container.pack(padx=10, pady=10)

        tk.Label(table_container, text="Day / Time", relief="ridge",
width=12).grid(row=0, column=0)
        for column_index, time_string in enumerate(time_header_labels):
            tk.Label(table_container, text=time_string, relief="ridge",
width=12).grid(row=0, column=column_index + 1)

        for row_index, day_name in enumerate(active_days_list):
            tk.Label(table_container, text=day_name, relief="ridge",
width=12).grid(row=row_index + 1, column=0)
            for column_index in range(total_slots_needed):
                slot_content = days_data[day_name][column_index]

                cell_text = "-"
                cell_color = "white"

```

```

        if slot_content is not None:
            subject_name, room_name = slot_content
            cell_text = subject_name + "\n" + room_name
            if "-L" in subject_name:
                cell_color = "#E1FFD7"
            else:
                cell_color = "#E1F2FF"

            cell_label = tk.Label(table_container, text=cell_text,
                                bg=cell_color, relief="solid",
                                borderwidth=1, width=12, height=3,
                                font=("Arial", 9))
            cell_label.grid(row=row_index + 1, column=column_index + 1)

root_window = tk.Tk()
app_instance = SimpleTimetableUI(root_window)
root_window.mainloop()

```

Output

Timetable Generator

SETTINGS

Exclude These Days:

☐ Mon

☐ Tue

☐ Wed

☐ Thu

☐ Fri

☒ Sat

☒ Sun

Day Starts At: 08:30

Day Ends At: 17:30

Slot Length (Minutes): 60

GENERATE SCHEDULE

Users can select the days to exclude, also start and end time of the day.

SCHEDULE FOR: BSAI-1

Day / Time	08:30	09:30	10:30	11:30	12:30	13:30	14:30	15:30	16:30
Mon	CP-L Lab 5	CP-L Lab 5	CP-L Lab 5	PPE E-106	PPE E-106	-	Tawheed E-103	ICT E-311	ICT E-311
Tue	IS E-106	IS E-106	AP E-202	-	-	ICT-L Lab 4	ICT-L Lab 4	ICT-L Lab 4	-
Wed	AP-L(A) AP Lab	AP-L(A) AP Lab	AP-L(A) AP Lab	AP E-210	AP E-210	-	DM E-210	-	CP E-207
Thu	-	-	-	-	-	-	-	-	-
Fri	AP-L(B) AP Lab	AP-L(B) AP Lab	AP-L(B) AP Lab	DM E-104	DM E-104	CP E-211	CP E-211	ICT E-211	-

SCHEDULE FOR: BSAI-2

Day / Time	08:30	09:30	10:30	11:30	12:30	13:30	14:30	15:30	16:30
Mon	OOP-L Lab 2	OOP-L Lab 2	OOP-L Lab 2	PS E-306	PS E-306	AC&AG E-307	DD E-307	FE E-209	FE E-209
Tue	FE E-202	-	DD-L(A) DLD Lab	DD-L(A) DLD Lab	DD-L(A) DLD Lab	PST E-205	PST E-205	OOP E-108	-
Wed	AC&AG E-111	AC&AG E-111	UQ-1 E-302	-	-	DD E-101	DD E-101	-	-
Thu	DD-L(B) DLD Lab	DD-L(B) DLD Lab	DD-L(B) DLD Lab	PS E-103	-	-	OOP E-202	OOP E-202	-

SCHEDULE FOR: BSAI-3

Day / Time	08:30	09:30	10:30	11:30	12:30	13:30	14:30	15:30	16:30
Mon	-	-	-	-	-	-	-	-	-
Tue	-	-	LA E-208	LA E-208	DSA E-211	-	UQ-2 E-308	-	-
Wed	CS E-205	CS E-205	PN E-102	PN E-102	LA E-105	DSA E-107	DSA E-107	-	-
Thu	-	PA-L Lab 2	PA-L Lab 2	PA-L Lab 2	-	COAL E-204	COAL E-204	-	-
Fri	COAL-L Lab 5	COAL-L Lab 5	COAL-L Lab 5	PN E-101	DSA-L Lab 4	DSA-L Lab 4	DSA-L Lab 4	CCE E-205	CCE E-205

SCHEDULE FOR: BSAI-4

Day / Time	08:30	09:30	10:30	11:30	12:30	13:30	14:30	15:30	16:30
Mon	DEMS-L Web Lab	DEMS-L Web Lab	DEMS-L Web Lab	DEMS E-107	DEMS E-107	-	-	UQ-3 E-307	-
Tue	DCN E-103	DCN E-103	DEMS E-307	-	DCN-L(A) CN Lab	DCN-L(A) CN Lab	DCN-L(A) CN Lab	ENT E-302	ENT E-302
Wed	-	-	CT E-301	CT E-301	-	DCN E-305	-	AI E-208	-
Thu	-	-	AI E-209	AI E-209	AHL AI Lab	AHL AI Lab	AHL AI Lab	-	CT E-209
Fri	DCN-L(B) CN Lab	DCN-L(B) CN Lab	DCN-L(B) CN Lab	-	-	-	-	-	-

A timetable for all classes will be generated depending on the settings chosen by the user.